

Uber Interview Preparation — Database & SQL Roles

1. Company Profile

Engineering Culture

- **Reliability obsession:** ride failures directly impact revenue and safety
- Real-time everywhere: surge pricing, driver matching, ETAs — all need sub-second DB responses
- Microservices-heavy architecture with strong data ownership per service
- Open source contributors: Schemaless (Uber's MySQL-backed document store), M3 (time-series), Cadence (workflow)
- Geo-awareness is fundamental — nearly every problem has a spatial dimension
- Post-incident culture: blameless retrospectives, rigorous runbooks

Database Technology Stack

Layer	Technology
Core OLTP	MySQL (Schemaless, custom MySQL-based document store)
PostgreSQL	Used for analytics, financial systems, internal tools
Time Series	M3DB (Uber's custom TSDB), InfluxDB
Geo/Spatial	PostGIS on PostgreSQL, custom H3 geo-indexing
Analytics	Apache Hive, Presto, Apache Spark
Message Queue	Apache Kafka
Cache	Redis
Search	Elasticsearch
Graph	Internal graph services
Data Warehouse	Apache Hive on S3

What Uber Values in DB/SQL Candidates

- Geospatial SQL is **not optional** — must know PostGIS or H3
 - Real-time performance: designed for P99 latency, not just average
 - Financial accuracy: fares, payouts, surge — must be exactly correct
 - Distributed transactions: eventual consistency is often not acceptable for money
 - Incident mentality: designs must include failure modes and fallbacks
-

2. Interview Process

Typical Rounds (SWE II / SWE III / Staff)

Round	Format	Duration	Focus
-------	--------	----------	-------

Technical Screen	HackerRank	90 min	SQL + coding
Technical Phone	CoderPad	60 min	SQL + design
Onsite 1	CoderPad	60 min	SQL + geospatial
Onsite 2	Whiteboard/Video	60 min	System design
Onsite 3	Whiteboard/Video	60 min	DB design + schema
Onsite 4	Video	45 min	Behavioral

Level Expectations

Level	Geo SQL	Real-time DB	System Design
SWE I (L3)	Basic ST_ functions	Awareness	Not required
SWE II (L4)	Joins + geo filters	Index design	Service-level
SWE III (L5)	Complex spatial queries	Performance tuning	Full system
Staff (L6)	Novel spatial algorithms	Distributed transactions	Platform-level

3. Geospatial Queries with PostGIS — 15 Real Examples

Setup:

```

CREATE EXTENSION postgis;
CREATE EXTENSION postgis_topology;
CREATE EXTENSION h3; -- Uber's H3 hexagonal geo-indexing

-- Core tables
CREATE TABLE drivers (
  driver_id      BIGSERIAL PRIMARY KEY,
  name           VARCHAR(200),
  license_plate  VARCHAR(20),
  vehicle_type   VARCHAR(30),
  is_available   BOOLEAN DEFAULT FALSE,
  current_location GEOMETRY(POINT, 4326),
  last_ping      TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE trips (
  trip_id        BIGSERIAL PRIMARY KEY,
  rider_id       BIGINT NOT NULL,
  driver_id      BIGINT,
  pickup_location GEOMETRY(POINT, 4326) NOT NULL,
  dropoff_location GEOMETRY(POINT, 4326),
  pickup_address TEXT,
  dropoff_address TEXT,
  status         VARCHAR(20) DEFAULT 'requested',

```

```

requested_at    TIMESTAMPTZ DEFAULT NOW(),
started_at      TIMESTAMPTZ,
completed_at    TIMESTAMPTZ,
fare_amount     NUMERIC(10,2),
distance_km     NUMERIC(8,3),
route_path      GEOMETRY(LINESTRING, 4326)
);

-- Spatial indexes (GIST is required for geo queries)
CREATE INDEX idx_drivers_location ON drivers USING GIST (current_location);
CREATE INDEX idx_trips_pickup ON trips USING GIST (pickup_location);

```

Geo Query 1: Find all available drivers within 2km of a pickup point.

```

SELECT
  driver_id,
  name,
  vehicle_type,
  ST_Distance(
    current_location::geography,
    ST_SetSRID(ST_MakePoint(-73.9857, 40.7484), 4326)::geography
  ) AS distance_m
FROM drivers
WHERE is_available = TRUE
  AND ST_DWithin(
    current_location::geography,
    ST_SetSRID(ST_MakePoint(-73.9857, 40.7484), 4326)::geography,
    2000 -- 2000 meters
  )
ORDER BY distance_m
LIMIT 10;

```

Geo Query 2: Calculate straight-line distance vs. actual route distance for trips.

```

SELECT
  trip_id,
  ST_Distance(pickup_location::geography, dropoff_location::geography) / 1000 AS
straight_km,
  distance_km AS actual_km,
  ROUND((distance_km / NULLIF(ST_Distance(pickup_location::geography,
dropoff_location::geography) / 1000, 0))::numeric, 2) AS detour_factor
FROM trips
WHERE status = 'completed'
  AND completed_at >= NOW() - INTERVAL '7 days';

```

Geo Query 3: Count trips that started in each city district (polygon-based).

```

SELECT
  d.district_name,
  COUNT(t.trip_id) AS trip_count,

```

```

    AVG(t.fare_amount) AS avg_fare
FROM districts d
JOIN trips t ON ST_Within(t.pickup_location, d.boundary)
WHERE t.requested_at >= NOW() - INTERVAL '24 hours'
GROUP BY d.district_name
ORDER BY trip_count DESC;

```

Geo Query 4: Find surge zones — areas with > 3x normal trip density.

```

WITH cell_counts AS (
    SELECT
        H3_LATLON_TO_CELL(ST_Y(pickup_location), ST_X(pickup_location), 8) AS
h3_cell,
        COUNT(*) AS current_trips
    FROM trips
    WHERE requested_at >= NOW() - INTERVAL '15 minutes'
    GROUP BY h3_cell
),
baseline AS (
    SELECT
        H3_LATLON_TO_CELL(ST_Y(pickup_location), ST_X(pickup_location), 8) AS
h3_cell,
        COUNT(*) / 4.0 AS baseline_trips -- 4 weeks / 15min windows
    FROM trips
    WHERE EXTRACT(DOW FROM requested_at) = EXTRACT(DOW FROM NOW())
        AND requested_at::time BETWEEN (NOW() - INTERVAL '15 minutes')::time
            AND NOW()::time
        AND requested_at >= NOW() - INTERVAL '28 days'
    GROUP BY h3_cell
)
SELECT
    cc.h3_cell,
    cc.current_trips,
    b.baseline_trips,
    ROUND(cc.current_trips / NULLIF(b.baseline_trips, 0), 2) AS surge_multiplier,
    H3_CELL_TO_BOUNDARY(cc.h3_cell) AS cell_boundary
FROM cell_counts cc
JOIN baseline b USING (h3_cell)
WHERE cc.current_trips > b.baseline_trips * 3;

```

Geo Query 5: Find drivers who frequently work in multiple cities (potential fraud detection).

```

SELECT
    driver_id,
    COUNT(DISTINCT city_id) AS cities_worked,
    ARRAY_AGG(DISTINCT city_name) AS city_list
FROM (
    SELECT
        t.driver_id,
        c.city_id,
        c.city_name

```

```

FROM trips t
JOIN cities c ON ST_Within(t.pickup_location, c.boundary)
WHERE t.started_at >= NOW() - INTERVAL '7 days'
) driver_cities
GROUP BY driver_id
HAVING COUNT(DISTINCT city_id) >= 3;

```

Geo Query 6: Calculate average speed per trip segment.

```

SELECT
  trip_id,
  ST_Length(route_path::geography) / 1000 AS route_km,
  EXTRACT(EPOCH FROM (completed_at - started_at)) / 3600 AS hours,
  ROUND(
    (ST_Length(route_path::geography) / 1000) /
    NULLIF(EXTRACT(EPOCH FROM (completed_at - started_at)) / 3600, 0)
    , 1) AS avg_speed_kmh
FROM trips
WHERE status = 'completed'
AND route_path IS NOT NULL;

```

Geo Query 7: Find the nearest airport to each driver's current location.

```

SELECT DISTINCT ON (d.driver_id)
  d.driver_id,
  a.airport_name,
  ST_Distance(d.current_location::geography, a.location::geography) AS dist_m
FROM drivers d
CROSS JOIN LATERAL (
  SELECT airport_name, location
  FROM airports
  ORDER BY d.current_location <-> location
  LIMIT 1
) a
WHERE d.is_available = TRUE;

```

Geo Query 8: Heatmap data — bucket trips into H3 hexagons at resolution 9.

```

SELECT
  H3_LATLON_TO_CELL(
    ST_Y(pickup_location),
    ST_X(pickup_location),
    9 -- ~174m hexagons
  ) AS h3_cell,
  COUNT(*) AS trip_count,
  AVG(fare_amount) AS avg_fare,
  AVG(EXTRACT(EPOCH FROM (started_at - requested_at))) AS avg_wait_s
FROM trips
WHERE requested_at >= NOW() - INTERVAL '24 hours'

```

```

    AND status IN ('completed', 'started')
GROUP BY h3_cell;

```

Geo Query 9: Find trips where the driver took a significantly longer route (detour detection).

```

SELECT
    trip_id,
    driver_id,
    rider_id,
    ST_Distance(pickup_location::geography, dropoff_location::geography) / 1000 AS
straight_km,
    distance_km AS actual_km,
    fare_amount
FROM trips
WHERE status = 'completed'
    AND distance_km > ST_Distance(pickup_location::geography,
dropoff_location::geography) / 1000 * 2
    AND distance_km > 5 -- meaningful distance
ORDER BY (distance_km / (ST_Distance(pickup_location::geography,
dropoff_location::geography) / 1000)) DESC;

```

Geo Query 10: Calculate driver utilization — time en route vs. idle.

```

SELECT
    driver_id,
    SUM(EXTRACT(EPOCH FROM (completed_at - started_at))) / 3600 AS driving_hours,
    SUM(EXTRACT(EPOCH FROM (started_at - assigned_at))) / 3600 AS
en_route_to_pickup_hours,
    COUNT(*) AS trips_completed,
    AVG(fare_amount) AS avg_fare
FROM trips
WHERE status = 'completed'
    AND completed_at >= NOW() - INTERVAL '7 days'
GROUP BY driver_id
ORDER BY driving_hours DESC;

```

Geo Query 11: Identify "dead zones" — areas with high demand but low driver supply.

```

WITH demand AS (
    SELECT
        H3_LATLON_TO_CELL(ST_Y(pickup_location), ST_X(pickup_location), 7) AS
h3_cell,
        COUNT(*) AS request_count
    FROM trips
    WHERE requested_at >= NOW() - INTERVAL '30 minutes'
    GROUP BY h3_cell
),
supply AS (
    SELECT
        H3_LATLON_TO_CELL(ST_Y(current_location), ST_X(current_location), 7) AS

```

```

h3_cell,
    COUNT(*) AS available_drivers
FROM drivers
WHERE is_available = TRUE
    AND last_ping >= NOW() - INTERVAL '5 minutes'
GROUP BY h3_cell
)
SELECT
    d.h3_cell,
    d.request_count,
    COALESCE(s.available_drivers, 0) AS available_drivers,
    d.request_count::numeric / NULLIF(COALESCE(s.available_drivers, 0), 0) AS
demand_supply_ratio,
    H3_CELL_TO_BOUNDARY(d.h3_cell) AS hex_boundary
FROM demand d
LEFT JOIN supply s USING (h3_cell)
WHERE COALESCE(s.available_drivers, 0) < d.request_count / 3
ORDER BY demand_supply_ratio DESC NULLS FIRST;

```

Geo Query 12: Calculate the centroid of all completed trips per city district.

```

SELECT
    district_name,
    ST_AsText(ST_Centroid(ST_Collect(pickup_location))) AS avg_pickup_centroid,
    ST_AsText(ST_Centroid(ST_Collect(dropoff_location))) AS avg_dropoff_centroid,
    COUNT(*) AS trip_count
FROM trips t
JOIN districts d ON ST_Within(t.pickup_location, d.boundary)
WHERE t.status = 'completed'
    AND t.started_at >= NOW() - INTERVAL '30 days'
GROUP BY district_name;

```

Geo Query 13: Find all trips that crossed a specific boundary (city/zone border).

```

SELECT t.trip_id, t.driver_id, t.fare_amount
FROM trips t
JOIN city_boundaries cb ON ST_Crosses(t.route_path, cb.boundary_line)
WHERE cb.boundary_name = 'NYC-NJ'
    AND t.status = 'completed';

```

Geo Query 14: Calculate trip density by time of day for each borough.

```

SELECT
    b.borough_name,
    EXTRACT(HOUR FROM t.requested_at) AS hour_of_day,
    COUNT(*) AS trips,
    AVG(t.fare_amount) AS avg_fare
FROM trips t
JOIN boroughs b ON ST_Within(t.pickup_location, b.boundary)
WHERE t.requested_at >= NOW() - INTERVAL '30 days'

```

```
GROUP BY b.borough_name, EXTRACT(HOUR FROM t.requested_at)
ORDER BY b.borough_name, hour_of_day;
```

Geo Query 15: Snap driver GPS coordinates to nearest road segment.

```
-- Using pgRouting (road network)
SELECT
    driver_id,
    last_ping,
    current_location,
    (SELECT geom FROM road_network ORDER BY current_location <-> geom LIMIT 1) AS
snapped_location,
    ST_Distance(
        current_location::geography,
        (SELECT geom FROM road_network ORDER BY current_location <-> geom LIMIT
1)::geography
    ) AS snap_distance_m
FROM drivers
WHERE is_available = TRUE
    AND last_ping >= NOW() - INTERVAL '30 seconds';
```

4. SQL Coding Round (Non-Geo)

Q1. Calculate driver earnings including bonuses for completing 10+ rides in a day.

```
SELECT
    driver_id,
    DATE(completed_at) AS work_date,
    COUNT(*) AS trips_completed,
    SUM(driver_payout) AS base_earnings,
    CASE WHEN COUNT(*) >= 10 THEN 25.00 ELSE 0 END AS daily_bonus,
    SUM(driver_payout) + CASE WHEN COUNT(*) >= 10 THEN 25.00 ELSE 0 END AS
total_earnings
FROM trips
WHERE status = 'completed'
GROUP BY driver_id, DATE(completed_at)
ORDER BY work_date, total_earnings DESC;
```

Q2. Find the optimal pickup time for each city — minimize rider wait time.

```
SELECT
    city_id,
    EXTRACT(HOUR FROM requested_at) AS hour,
    EXTRACT(DOW FROM requested_at) AS day_of_week,
    ROUND(AVG(EXTRACT(EPOCH FROM (started_at - requested_at))) / 60, 1) AS
avg_wait_min,
    COUNT(*) AS trip_count
FROM trips t
JOIN cities c ON ST_Within(t.pickup_location, c.boundary)
```



```

WHERE status = 'completed'
  AND started_at IS NOT NULL
GROUP BY city_id, hour, day_of_week
ORDER BY avg_wait_min;

```

Q3. Surge pricing trigger: calculate demand/supply ratio per zone.

```

WITH zone_demand AS (
  SELECT zone_id, COUNT(*) AS requests_15min
  FROM trip_requests r
  JOIN zones z ON ST_Within(r.pickup_location, z.boundary)
  WHERE r.requested_at >= NOW() - INTERVAL '15 minutes'
  GROUP BY zone_id
),
zone_supply AS (
  SELECT zone_id, COUNT(*) AS available_drivers
  FROM drivers d
  JOIN zones z ON ST_Within(d.current_location, z.boundary)
  WHERE d.is_available = TRUE AND d.last_ping >= NOW() - INTERVAL '2 minutes'
  GROUP BY zone_id
)
SELECT
  z.zone_id,
  z.zone_name,
  COALESCE(zd.requests_15min, 0) AS demand,
  COALESCE(zs.available_drivers, 0) AS supply,
  GREATEST(1.0,
    LEAST(5.0,
      ROUND((COALESCE(zd.requests_15min, 0)::numeric /
        NULLIF(COALESCE(zs.available_drivers, 1), 0)) * 0.5 + 0.5, 1)
    )
  ) AS surge_multiplier
FROM zones z
LEFT JOIN zone_demand zd USING (zone_id)
LEFT JOIN zone_supply zs USING (zone_id);

```

Q4. Month-end driver settlement: calculate total due with deductions.

```

SELECT
  driver_id,
  SUM(fare_amount) AS gross_fares,
  SUM(fare_amount * 0.25) AS uber_commission,
  SUM(fare_amount * 0.75) AS driver_gross,
  SUM(tip_amount) AS tips,
  SUM(surge_amount) AS surge_earnings,
  SUM(fare_amount * 0.75 + tip_amount + surge_amount) AS total_earnings,
  SUM(CASE WHEN promo_applied THEN promo_amount ELSE 0 END) AS promo_costs,
  SUM(fare_amount * 0.75 + tip_amount + surge_amount) -
    SUM(CASE WHEN promo_applied THEN promo_amount ELSE 0 END) AS net_settlement
FROM trips
WHERE status = 'completed'

```

```

    AND DATE_TRUNC('month', completed_at) = DATE_TRUNC('month', NOW() - INTERVAL '1
month')
GROUP BY driver_id;

```

Q5. Find the top 10 busiest driver-rider city pair combinations.

```

SELECT
    pickup_city,
    dropoff_city,
    COUNT(*) AS trip_count,
    ROUND(AVG(distance_km), 2) AS avg_distance_km,
    ROUND(AVG(fare_amount), 2) AS avg_fare
FROM trips t
JOIN LATERAL (SELECT city_name AS pickup_city FROM cities WHERE
ST_Within(t.pickup_location, boundary) LIMIT 1) pu ON TRUE
JOIN LATERAL (SELECT city_name AS dropoff_city FROM cities WHERE
ST_Within(t.dropoff_location, boundary) LIMIT 1) dr ON TRUE
WHERE status = 'completed'
    AND started_at >= NOW() - INTERVAL '30 days'
GROUP BY pickup_city, dropoff_city
ORDER BY trip_count DESC
LIMIT 10;

```

5. Database Design Tasks

Task 1: Design Uber's Surge Pricing System

```

CREATE TABLE surge_configs (
    zone_id          INT NOT NULL,
    multiplier        NUMERIC(4,2) NOT NULL DEFAULT 1.0,
    effective_from    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    effective_to      TIMESTAMPTZ,
    reason            VARCHAR(50), -- 'demand', 'weather', 'event'
    PRIMARY KEY (zone_id, effective_from)
);

-- Current surge view
CREATE VIEW current_surge AS
SELECT DISTINCT ON (zone_id)
    zone_id, multiplier, effective_from
FROM surge_configs
WHERE effective_from <= NOW() AND (effective_to IS NULL OR effective_to > NOW())
ORDER BY zone_id, effective_from DESC;

-- Surge calculation trigger
CREATE OR REPLACE FUNCTION recalculate_surge(p_zone_id INT) RETURNS void AS $$
DECLARE
    v_demand INT;
    v_supply INT;

```

```

    v_multiplier NUMERIC(4,2);
BEGIN
    SELECT COUNT(*) INTO v_demand
    FROM trip_requests
    WHERE zone_id = p_zone_id AND requested_at >= NOW() - INTERVAL '15 minutes';

    SELECT COUNT(*) INTO v_supply
    FROM drivers
    WHERE zone_id = p_zone_id AND is_available = TRUE;

    v_multiplier := GREATEST(1.0, LEAST(5.0, ROUND((v_demand::numeric /
    NULLIF(v_supply, 1)) * 0.5 + 0.5, 1)));

    INSERT INTO surge_configs (zone_id, multiplier)
    VALUES (p_zone_id, v_multiplier)
    ON CONFLICT (zone_id, effective_from) DO NOTHING;
END;
$$ LANGUAGE plpgsql;

```

Task 2: Design Driver-Rider Matching Schema

```

CREATE TABLE matching_sessions (
    session_id      UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    trip_request_id BIGINT NOT NULL UNIQUE,
    rider_id        BIGINT NOT NULL,
    status          VARCHAR(20) DEFAULT 'matching',
    started_at      TIMESTAMPTZ DEFAULT NOW(),
    matched_at      TIMESTAMPTZ,
    algorithm_version VARCHAR(20)
);

CREATE TABLE driver_candidates (
    session_id      UUID REFERENCES matching_sessions(session_id),
    driver_id       BIGINT NOT NULL,
    distance_m      NUMERIC(10,2),
    eta_seconds     INT,
    score           NUMERIC(8,5), -- matching score
    offered_at      TIMESTAMPTZ DEFAULT NOW(),
    response        VARCHAR(20), -- 'accepted', 'declined', 'timeout'
    responded_at    TIMESTAMPTZ,
    PRIMARY KEY (session_id, driver_id)
);

-- Driver matching function
CREATE INDEX idx_drivers_available_geo ON drivers USING GIST (current_location)
WHERE is_available = TRUE AND last_ping >= NOW() - INTERVAL '30 seconds';

```

Task 3: Design Fare Calculation and Dispute Resolution

```

CREATE TABLE fare_components (
  fare_id          BIGSERIAL PRIMARY KEY,
  trip_id          BIGINT UNIQUE NOT NULL,
  base_fare        NUMERIC(10,2),
  distance_fare    NUMERIC(10,2),
  time_fare        NUMERIC(10,2),
  surge_multiplier NUMERIC(4,2) DEFAULT 1.0,
  surge_amount     NUMERIC(10,2),
  booking_fee      NUMERIC(10,2),
  toll_amount      NUMERIC(10,2) DEFAULT 0,
  tip_amount       NUMERIC(10,2) DEFAULT 0,
  promo_discount   NUMERIC(10,2) DEFAULT 0,
  total_fare       NUMERIC(10,2) GENERATED ALWAYS AS
    ((base_fare + distance_fare + time_fare) * surge_multiplier + booking_fee +
    toll_amount + tip_amount - promo_discount) STORED,
  calculated_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE fare_disputes (
  dispute_id       BIGSERIAL PRIMARY KEY,
  trip_id          BIGINT NOT NULL,
  user_id          BIGINT NOT NULL,
  user_type        VARCHAR(10) CHECK (user_type IN ('rider','driver')),
  dispute_type     VARCHAR(50),
  original_amount  NUMERIC(10,2),
  disputed_amount  NUMERIC(10,2),
  resolution       NUMERIC(10,2),
  status           VARCHAR(20) DEFAULT 'open',
  created_at       TIMESTAMPTZ DEFAULT NOW(),
  resolved_at      TIMESTAMPTZ
);

```

6. System Design with DB Focus

Scenario 1: Design Real-Time Driver Location Tracking

Requirements: 5M active drivers, location updates every 5 seconds.

Architecture:

```

Driver App → WebSocket → Location Service →
  Write: Redis GEOADD (real-time, 5s TTL per driver)
        Kafka (async persistence)
          → Consumer → PostgreSQL (INSERT, 30s batch)
          → Consumer → H3 Cell assignment → Redis Sorted Set per cell

```

Geo Query Path:

```

Rider requests pickup → Zone H3 cell lookup →
  Redis GEORADIUS (< 5ms) → top 10 drivers by distance + availability

```

Schema (PostgreSQL for historical):

```
driver_locations (driver_id, location GEOMETRY, recorded_at TIMESTAMPTZ)
Partitioned by day, BRIN index on recorded_at
Retention: 30 days raw, 1 year hourly aggregated
```

Scenario 2: Design Surge Pricing at City Scale

Requirements: Recalculate surge every 60 seconds for 5,000 zones.

Approach:

1. Kafka streams of trip requests and driver location updates
2. Flink job: aggregate demand/supply per H3 cell every 60s
3. Write surge multipliers to Redis hash `surge:{city_id}:{h3_cell}`
4. API reads surge from Redis (< 1ms)
5. PostgreSQL: audit trail of all surge changes (for regulatory compliance)

Scenario 3: Design Uber's Financial Reconciliation System

Requirements: Every dollar must be accounted for. No missing or double-counted transactions.

```
-- Ledger-style accounting
CREATE TABLE financial_ledger (
  ledger_id      BIGSERIAL PRIMARY KEY,
  trip_id        BIGINT,
  entry_type      VARCHAR(50), --
  'fare_charge', 'driver_payout', 'uber_take', 'promo', 'refund', 'chargeback'
  debit_account  VARCHAR(50),
  credit_account VARCHAR(50),
  amount         NUMERIC(12,4) NOT NULL,
  currency       CHAR(3) DEFAULT 'USD',
  reference_id   UUID NOT NULL,
  created_at     TIMESTAMPTZ DEFAULT NOW(),
  CONSTRAINT positive_amount CHECK (amount > 0)
);

-- Daily reconciliation check
SELECT
  SUM(CASE WHEN debit_account LIKE 'rider%' THEN amount END) AS
total_charged_to_riders,
  SUM(CASE WHEN credit_account LIKE 'driver%' THEN amount END) AS
total_paid_to_drivers,
  SUM(CASE WHEN credit_account = 'uber_revenue' THEN amount END) AS
uber_net_revenue
FROM financial_ledger
WHERE DATE(created_at) = CURRENT_DATE - 1;
```

7. Behavioral Questions

Real-Time Systems

- "Tell me about a time you optimized a database query that was causing real-time latency issues."
- "Describe a production incident involving the database. How did you diagnose and resolve it?"

- "How did you handle a situation where database performance degraded unexpectedly at peak hours?"

Reliability

- "Tell me about designing for failure. How have you built database systems that degrade gracefully?"
- "Describe a migration you planned and executed without downtime."

Geospatial / Technical

- "Have you worked with spatial databases before? Walk me through a geospatial query you wrote."

8. Mock Interview Script

Problem: "Design the database for Uber Eats order management."

Interviewer: "Walk me through how you'd design the database for Uber Eats — restaurants, menus, orders, delivery tracking."

Candidate response:

"Before designing, a few questions: What's the scale? (50M orders/day) Are menus multi-language? (Yes) Do we need delivery location tracking? (Yes, real-time GPS)"

```
-- Core schema
CREATE TABLE restaurants (
  restaurant_id BIGSERIAL PRIMARY KEY,
  name          VARCHAR(500),
  location      GEOMETRY(POINT, 4326) NOT NULL,
  delivery_radius_m INT DEFAULT 5000,
  is_open       BOOLEAN DEFAULT FALSE,
  avg_prep_min  INT
);

CREATE TABLE menu_items (
  item_id        BIGSERIAL PRIMARY KEY,
  restaurant_id  BIGINT REFERENCES restaurants(restaurant_id),
  name_translations JSONB, -- {"en": "Burger", "es": "Hamburguesa"}
  price          NUMERIC(8,2),
  category       VARCHAR(100),
  is_available   BOOLEAN DEFAULT TRUE,
  prep_time_min  INT
);

CREATE TABLE orders (
  order_id       BIGSERIAL PRIMARY KEY,
  rider_id       BIGINT NOT NULL,
  restaurant_id  BIGINT NOT NULL,
  delivery_address JSONB,
  delivery_location GEOMETRY(POINT, 4326),
  status         VARCHAR(30) DEFAULT 'placed',
  total_amount   NUMERIC(10,2),
  placed_at      TIMESTAMPTZ DEFAULT NOW(),
  estimated_delivery TIMESTAMPTZ
```

```

) PARTITION BY RANGE (placed_at);

CREATE TABLE order_items (
  order_id      BIGINT NOT NULL,
  item_id       BIGINT NOT NULL,
  quantity      INT NOT NULL CHECK (quantity > 0),
  unit_price    NUMERIC(8,2) NOT NULL,
  special_instructions TEXT,
  PRIMARY KEY (order_id, item_id)
);

CREATE TABLE delivery_assignments (
  assignment_id BIGSERIAL PRIMARY KEY,
  order_id      BIGINT UNIQUE NOT NULL,
  courier_id    BIGINT NOT NULL,
  assigned_at   TIMESTAMPTZ DEFAULT NOW(),
  picked_up_at TIMESTAMPTZ,
  delivered_at  TIMESTAMPTZ,
  current_location GEOMETRY(POINT, 4326)
);

-- Find nearby open restaurants for a delivery address
CREATE INDEX idx_restaurants_geo ON restaurants USING GIST (location);
SELECT
  restaurant_id, name,
  ST_Distance(location::geography, :customer_point::geography) AS dist_m,
  avg_prep_min
FROM restaurants
WHERE is_open = TRUE
  AND ST_DWithin(location::geography, :customer_point::geography, delivery_radius_m)
ORDER BY dist_m;

```

9. Evaluation Rubric

SWE II (L4)

Signal	Pass	Exceeds
SQL	Complex queries correct	Geo-aware optimization
PostGIS	Basic ST_Distance, ST_DWithin	H3 integration, GIST indexes
Design	Core entities, FKs	Partitioning, real-time patterns
Systems	Identifies bottlenecks	Discusses cache + queue

SWE III (L5)

Signal	Pass	Exceeds
Geo SQL	Multi-step spatial queries	Custom spatial algorithms

Real-time	Understands Redis + PG split	Designs full data pipeline
Financial	Aware of precision/ACID needs	Ledger accounting patterns
Scale	10M trips/day reasoning	Sub-second P99 design

10. Red Flags

- Cannot explain what a GIST index is
- Uses Euclidean distance instead of geodetic (great circle) distance for geo queries
- Ignores PostGIS or claims to have never used spatial SQL
- No knowledge of H3 hexagonal indexing (major at Uber)
- Schema designs with no discussion of real-time latency
- Cannot discuss trade-offs between Redis and PostgreSQL for location data
- Ignores financial accuracy in fare/payout designs (no NUMERIC, no constraints)

11. Green Flags

- Immediately uses `ST_Distance` for accurate distance calculations
- Knows the difference between GEOMETRY and GEOGRAPHY in PostGIS
- Mentions H3 by name or discusses hexagonal grid indexing
- Designs include Redis for hot geospatial data with PostgreSQL for persistence
- Discusses P99 latency requirements in system design
- Mentions BRIN indexes for time-series location history
- Financial designs use NUMERIC (not FLOAT) for all money columns
- Discusses rate limiting and idempotency for fare calculations

12. Salary Bands (Approximate, USD, 2024–2025)

Level	Base	Annual Bonus	RSU (4yr)	Total Comp
L3	\$130K–\$160K	10%	\$60K–\$100K	\$160K–\$210K
L4	\$160K–\$200K	15%	\$120K–\$200K	\$210K–\$300K
L5	\$200K–\$250K	20%	\$200K–\$400K	\$300K–\$480K
L6	\$250K–\$320K	25%	\$400K–\$700K	\$500K–\$750K
L7	\$320K–\$400K	30%	\$700K–\$1.4M	\$850K–\$1.4M

San Francisco rates. NYC ~5% lower. Remote ~15–20% lower.

13. Preparation Timeline (4-Week Plan)

Week 1: PostGIS & Geospatial SQL

- Day 1–2: Install PostGIS, learn `ST_Distance`, `ST_DWithin`, `ST_Within`, `ST_Intersects`
- Day 3: Study GIST and SP-GIST indexes for geometry
- Day 4: Study H3 hexagonal grid (Uber's open-source, papers + GitHub)
- Day 5: Write 5 geospatial queries from scratch

- Day 6–7: Practice geo-SQL on real datasets (OpenStreetMap data)

Week 2: Uber-Specific Domains

- Day 1: Study surge pricing algorithm (Uber blog posts)
- Day 2: Driver-rider matching algorithms and schema design
- Day 3: Real-time location tracking architecture (Redis geospatial)
- Day 4: Financial systems: ledger accounting, idempotency
- Day 5: Fare calculation design with ACID requirements
- Day 6–7: Mock design interview: design Uber Eats from scratch

Week 3: System Design + SQL

- Day 1–2: Design real-time driver tracking system (full stack)
- Day 3–4: Kafka + PostgreSQL pipeline for trip events
- Day 5: BRIN indexes for time-series geo data
- Day 6–7: Hard SQL: session analysis, gap detection, financial aggregations

Week 4: Polish

- Day 1–2: Behavioral: reliability + incident stories
- Day 3: Review Uber's engineering blog (real-world SQL and DB patterns)
- Day 4–5: Timed mock interviews
- Day 6–7: Rest + final review of geospatial cheat sheet